homeowner. The activity diagram notes two prompts that are the responsibility of the interface—"prompt for reentry" and "prompt for another view." These prompts and the decisions associated with them fall within the Interface swimlane. However, arrows lead from that swimlane back to the **Homeowner** swimlane, where homeowner actions occur.

Use cases, along with the activity and swimlane diagrams, are procedurally oriented. Taken together they can be used to represent the way various actors invoke specific functions (or other procedural steps) to meet the requirements of the system.

8.6 SUMMARY

The objective of requirements modeling is to create a variety of representations that describe what the customer requires, establish a basis for the creation of a software design, and define a set of requirements that can be validated once the software is built. The requirements model bridges the gap between a system-level description that describes overall system and business functionality and a software design that describes the software's application architecture, user interface, and component-level structure.

Scenario-based models depict software requirements from the user's point of view. The use case—a narrative or template-driven description of an interaction between an actor and the software—is the primary modeling element. Derived during requirements elicitation, the use case defines the key steps for a specific function or interaction. The degree of use case formality and detail varies, but they can provide necessary input to all other analysis modeling activities. Scenarios can also be described using an activity diagram—a graphical representation that depicts the processing flow within a specific scenario. Temporal relations in a use case can be modeled using sequence diagrams.

Class-based modeling uses information derived from use cases and other written application descriptions to identify analysis classes. A grammatical parse may be used to extract candidate classes, attributes, and operations from text-based narratives. Criteria for the definition of a class are defined using the parse results.

A set of class-responsibility-collaborator index cards can be used to define relationships between classes. In addition, a variety of UML modeling notation can be applied to define hierarchies, relationships, associations, aggregations, and dependencies among classes.

Behavioral modeling during requirements analysis depicts dynamic behavior of the software. The behavioral model uses input from scenario-based or class-based elements to represent the states of analysis classes. To accomplish this, states are identified, the events that cause a class (or the system) to make a transition from one state to another are defined, and the actions that occur as transition is accomplished are also identified. UML state diagrams, activity diagrams, swim lane diagrams, and sequence diagrams can be used for behavioral modeling.

PROBLEMS AND POINTS TO PONDER

8.1. Is it possible to begin coding immediately after a requirements model has been created? Explain your answer, and then argue the counterpoint.

8.2. An analysis rule of thumb is that the model "should focus on requirements that are visible within the problem or business domain." What types of requirements are *not* visible in these domains? Provide a few examples.

8.3. The department of public works for a large city has decided to develop a Web-based pothole tracking and repair system (PHTRS). A description follows:

> Citizens can log onto a website and report the location and severity of potholes. As potholes are reported they are logged within a "public works department repair system" and are assigned an identifying number, stored by street address, size (on a scale of 1 to 10), location (middle, curb, etc.), district (determined from street address), and repair priority (determined from the size of the pothole). Work order data are associated with each pothole and include pothole location and size, repair crew identifying number, number of people on crew, equipment assigned, hours applied to repair, hole status (work in progress, repaired, temporary repair, not repaired), amount of filler material used, and cost of repair (computed from hours applied, number of people, material and equipment used). Finally, a damage file is created to hold information about reported damage due to the pothole and includes citizen's name, address, phone number, type of damage, and dollar amount of damage. PHTRS is an online system; all queries are to be made interactively.

Draw a UML use case diagram PHTRS system. You'll have to make a number of assumptions about the manner in which a user interacts with this system.

8.4. Write two or three use cases that describe the roles of various actors in the PHTRS described in Problem 8.3.

8.5. Develop an activity diagram for one aspect of PHTRS.

8.6. Develop a swimlane diagram for one or more aspects of PHTRS.

8.7. Develop a class model for the PHTRS system presented in Problem 8.3.

8.8. Develop a complete set of CRC model index cards on the product or system you chose as part of Problem 8.3.

8.9. Conduct a review of the CRC index cards with your colleagues. How many additional classes, responsibilities, and collaborators were added as a consequence of the review?

8.10. How does a sequence diagram differ from a state diagram? How are they similar?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

9 DESIGN CONCEPTS

Software design encompasses the set of principles, concepts, and practices that lead to the development of a high-quality system or product. *Design principles* establish an overriding philosophy that guides the design work you must perform. *Design concepts* must be understood before the mechanics of design practice are applied, and *design practice* leads to the creation of various representations of the software that serve as a guide for the construction activity that follows.

KEY CONCEPTS

abstraction 163
architecture 163
cohesion 167
data design 174
design modeling principles 173
design process 159
functional independence 167
good design 160
information hiding 166
modularity 165
patterns 164
quality attributes 160
quality guidelines 160
refactoring 168
separation of concerns 165
software design 157
stepwise refinement 167
technical debt 157

QUICK LOOK

What is it? Design is what almost every engineer wants to do. It is the place where creativity rules—where requirements and technical considerations come together in the formulation of a product or system. Design creates a representation or model of the software and provides detail about software architecture, data structures, interfaces, and components that are necessary to implement the system.

Who does it? Software engineers conduct each of the design tasks while continuing communication with the stakeholders.

Why is it important? During the design phase, you model the system or product that needs to be built. The design model can be assessed for quality and improved before code is generated, tests are conducted, and end users become involved in large numbers.

What are the steps? Design makes use of several different representations of the software. First, the architecture of the system or product must be modeled. Then, the interfaces that connect the software to end users, to other systems and devices, and to its own constituent components are represented. Finally, the software components that are used to construct the system are designed.

What is the work product? A design model that encompasses architectural, interface, component-level, and deployment representations is the primary work product that is produced during software design.

How do I ensure that I've done it right? The design model is assessed by the software team (including relevant stakeholders) in an effort to determine whether it contains errors, inconsistencies, or omissions; whether better alternatives exist; and whether the model can be implemented within the constraints, schedule, and cost that have been established.

Design is pivotal to successful software engineering. Some developers are tempted to begin programming once the use cases have been created, without regard to how the software components needed to implement the use cases relate to one another. It is possible to do analysis, design, and implementation iteratively by creating several software increments. It is a bad idea to ignore the design considerations needed to create an appropriate architecture for the evolving software product. *Technical debt* is a concept in software development that refers to costs associated with rework caused by choosing a "quick and dirty" solution right now instead of using a better approach that would take more time. It is impossible to avoid creating technical debt when building a software product incrementally. However, a good development team must try to pay down this technical debt by refactoring (Section 9.3.9) the software on a regular basis. Just like taking out a loan, you can wait until the loan is due and pay a lot of interest or you can pay the loan off a little at a time and pay less interest overall.

One strategy to keep technical debt in check without delaying coding is to make use of the design practices of diversification and convergence. *Diversification* is the practice of identifying possible design alternatives suggested by the elements of the requirements model. *Convergence* is the process of evaluating and rejecting design alternatives that do not meet the constraints imposed by the nonfunctional requirements defined for the software solution. Diversification and convergence combine (1) intuition and judgment based on experience in building similar entities, (2) a set of principles and/or heuristics that guide the way in which the model evolves, (3) a set of criteria that enables quality to be judged, and (4) a process of iteration that ultimately leads to a final design representation. Once a viable design alternative is identified this way, the developers are in a good position to create a software increment that is not likely to be a throwaway prototype.

Software design changes continually as new methods, better analysis, and broader understanding evolve.[1] Even today, most software design methodologies lack the depth, flexibility, and quantitative nature that are normally associated with more classical engineering design disciplines. However, methods for software design do exist, criteria for design quality are available, and design notation can be applied.

In this chapter, we explore the fundamental concepts and principles that are applicable to all software design, the elements of the design model, and the impact of patterns on the design process. In Chapters 10 through 14 we'll present a variety of software design methods as they are applied to architectural, interface, and component-level design as well as pattern-based, mobile, and user experience design approaches.

9.1 DESIGN WITHIN THE CONTEXT OF SOFTWARE ENGINEERING

Software design sits at the technical kernel of software engineering and is applied regardless of the software process model that is used. Beginning once software requirements have been analyzed and modeled, software design is the last software engineering action within the modeling activity and sets the stage for **construction** (code generation and testing).

1 Those readers with further interest in the philosophy of software design might have interest in Philippe Kruchen's intriguing discussion of "postmodern" design [Kru05].

Each of the elements of the requirements model (Chapter 8) provide information that is necessary to create the four design models required for a complete specification of design. The flow of information during software design is illustrated in Figure 9.1. The requirements model, manifested by scenario-based, class-based, and behavioral elements, feed the design task. Using design notation and design methods discussed in later chapters, design produces a data/class design, an architectural design, an interface design, and a component design.

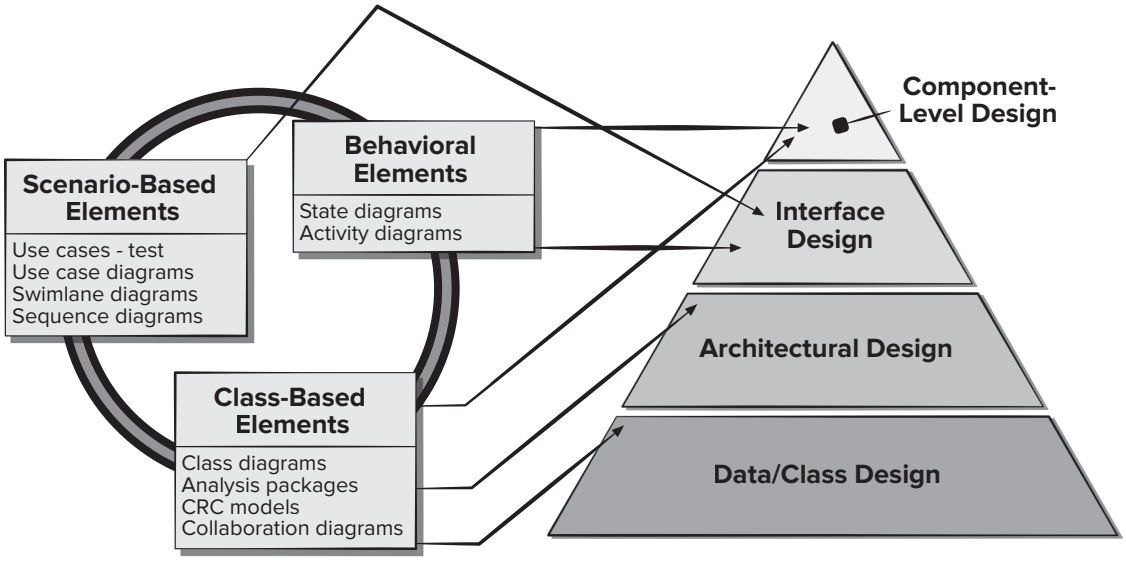
The data/class design transforms class models (Chapter 8) into design class realizations and the requisite data structures required to implement the software. The objects and relationships defined in the CRC model and the detailed data content depicted by class attributes and other notation provide the basis for the data design activity. Part of class design may occur in conjunction with the design of software architecture. More detailed class design occurs as each software component is designed.

The architectural design defines the relationship between major structural elements of the software, the architectural style, and patterns (Chapter 14) that can be used to achieve the requirements defined for the system, and the constraints that affect the way in which architecture can be implemented [Sha15]. The architectural design representation—the framework of a computer-based system—is derived from the requirements model.

The interface design describes how the software communicates with systems that interoperate with it, and with humans who use it. An interface implies a flow of information (e.g., data and/or control) and a specific type of behavior. Therefore, usage scenarios and behavioral models provide much of the information required for interface design.

The component-level design transforms structural elements of the software architecture into a procedural description of software components. Information obtained from the class-based models and behavioral models serve as the basis for component design.

FIGURE 9.1 **Translating the requirements model into the design model**



During design you make decisions that will ultimately affect the success of software construction and, just as important, the ease with which software can be maintained. But why is design so important?

The importance of software design can be stated with a single word—*quality*. Design is the place where quality is fostered in software engineering. It provides you with representations of software that can be assessed for quality. Design is the only way that you can accurately translate stakeholders' requirements into a finished software product or system. Software design serves as the foundation for all the software engineering and software support activities that follow. Without design, you risk building an unstable system—one that will fail when small changes are made; one that may be difficult to test; one whose quality cannot be assessed until late in the software process. Late in the project is when time is short, and many budgeted dollars have already been spent.

SafeHome



Design Versus Coding

The scene: Jamie's cubicle, as the team prepares to translate requirements into design.

The players: Jamie, Vinod, and Ed, all members of the *SafeHome* software engineering team.

The conversation:

Jamie: You know, Doug [the team manager] is obsessed with design. I gotta be honest, what I really love doing is coding. Give me C++ or Java, and I'm happy.

Ed: Nah . . . you like to design.

Jamie: You're not listening—coding is where it's at.

Vinod: I think what Ed means is that you don't really like coding; you like to design and express it in code. Code is the language you use to represent the design.

Jamie: And what's wrong with that?

Vinod: Level of abstraction.

Jamie: Huh?

Ed: A programming language is good for representing details like data structures and algorithms, but it's not so good for representing architecture or component-to-component collaboration . . . stuff like that.

Vinod: And a screwed-up architecture can ruin even the best code.

Jamie (thinking for a minute): So, you're saying that I can't represent architecture in code . . . that's not true.

Vinod: You can certainly imply architecture in code, but in most programming languages, it's difficult to get a quick, big-picture read on architecture by examining the code.

Ed: And that's what we want before we begin coding.

Jamie: Okay, maybe design and coding are different, but I still like coding better.

9.2 The Design Process

Software design is an iterative process through which requirements are translated into a "blueprint" for constructing the software. Initially, the blueprint depicts a holistic view of software. That is, the design is represented at a high level of abstraction—a level that can be directly traced to the specific system objective and more detailed data, functional, and behavioral requirements. As design iterations occur, subsequent